

Roots of Nonlinear Equations

CL249 18-08-2026

Roots of Nonlinear Equations

$$f(x) = 0$$

Eg.

$$x + \frac{1}{4x} = 15$$

$$x^3 + 4x^2 - 5 = 0$$

$$\sin^3(x) - 3x = 0$$

Bracketing Methods Overview

Method 1. Bisection Method

Root estimation relies purely on dividing the interval in half at every step, relying on the *Intermediate Value Theorem*:

$$X_m = \frac{a+b}{2} \quad f(a) \cdot f(b) < 0$$

Core Guarantee: Always converges if , but entirely ignores function magnitude or slope.

Method 2. Regula Falsi (False Position)

Improves upon Bisection by connecting endpoints with a linear secant line to estimate the root intercept:

$$X_c = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$$

Core Idea: Leverages function values for faster speed, but can suffer severe slowdowns in convex curves.

Bisection

$root = bisection(f, a, b, switch = 0, tol = 1.0e - 9)$

This finds root of $f(x) = 0$, roots are bracketed in $[a, b]$.

Switch: switch=1 returns root=None

Repeat till $|a - b| \leq \varepsilon$

Each step Δx is reduced to $\Delta x/2$

After n steps: $\frac{\Delta x}{2^n} = \varepsilon$

So, max steps $n = \frac{\ln(\frac{\Delta x}{\varepsilon})}{\ln(2)}$

Algorithm for Bisection

1. Define the function $f(x)$
2. Choose initial guesses a and b such that $f(a) * f(b) < 0$
3. Set the allowed error (tolerance), ϵ
4. Repeat until $(b - a)/2 < \epsilon$ or $|f(c)| < \epsilon$:
 - (i) Compute $c = (a + b) / 2$
 - (ii) If $f(c) == 0$ (or $|f(c)| < \epsilon$), return c as the root
Else if $f(c) * f(a) < 0$: $b = c$
Else: $a = c$
5. Return c as the approximate root

```
import math
```

```
import numpy as np
```

```
from numpy import sign
```

```
def bisection(f,x1,x2,switch=1,tol=1.0e-9):
```

```
    f1=f(x1)
```

```
    if abs(f1)<= tol: return x1
```

```
    f2=f(x2)
```

```
    if abs(f2)<= tol: return x2
```

```
    if sign(f1) == sign(f2):
```

```
        print('Root is not bracketed')
```

```
n=int(math.ceil(math.log(abs(x2-x1)/tol)/math.log(2.0)))
```

```
for i in range(n):
```

```
    x3=0.5*(x1+x2)
```

```
    f3=f(x3)
```

```
    if (switch == 1) and (abs(f3) > abs(f1)) and (abs(f3) >  
abs(f2)):
```

```
        return None
```

```
    if abs(f3) <= tol:
```

```
        return x3
```

```
    if sign(f2) != sign(f3):
```

```
        x1=x3
```

```
        f1=f3
```

```
    else:
```

```
        x2=x3;f2=f3
```

```
    return (x1+x2)/2.0
```

Bisection Method Drawbacks

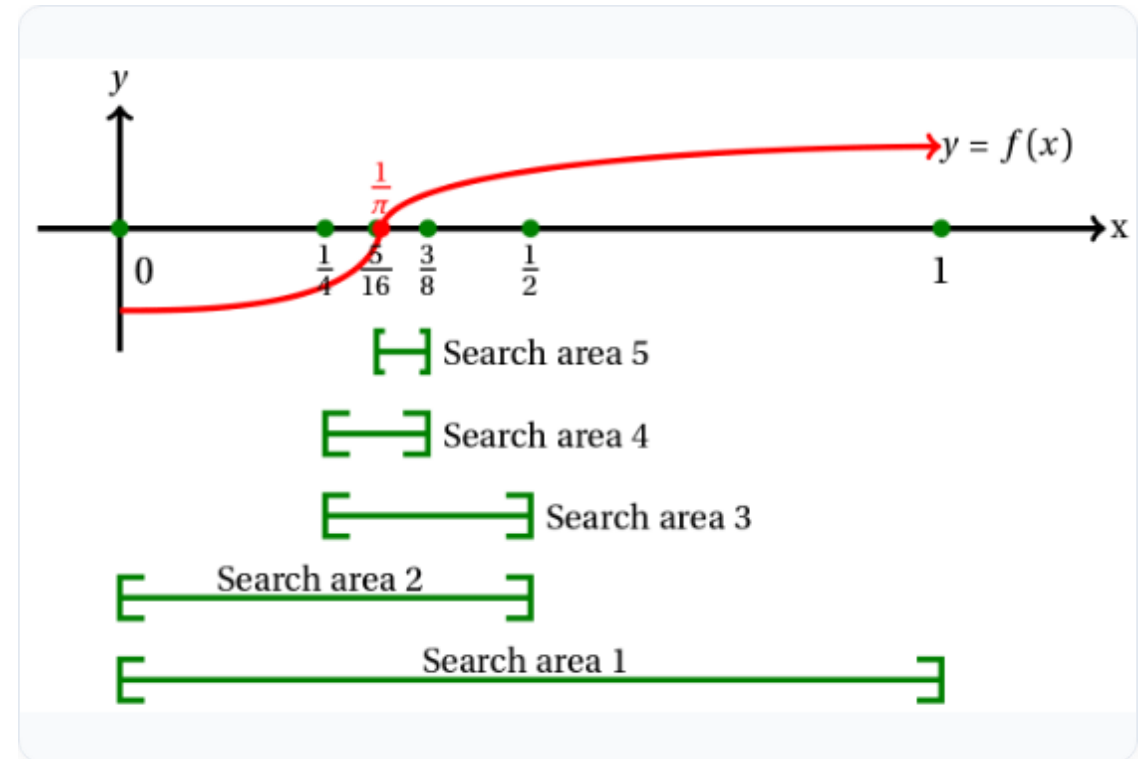
Slow Linear Convergence Rate: Reduces error by a constant factor of 0.5 per iteration ($E_{n+1} = 0.5E_n$), requiring roughly 3.3 iterations per decimal digit of accuracy.

Blind to Function Geometry: Uses only signs of , ignoring whether the function is nearly flat or extremely steep near the root. $f(a) \cdot f(b) > 0$

Inability to Detect Even Multiplicity: Fails completely when roots touch the x-axis without crossing (e.g.,), as .

$$f(x) = x^2$$

High Computation Cost: Requires many evaluations for high-precision engineering models compared to Newton-Raphson or Secant.



Regula falsi

Bracketing method: Start with two guesses a, b such that $f(a)$ and $f(b)$ have opposite signs

Formula: The root is approximated using Linear Interpolation using the formula:

$$c = a - \frac{f(a) \cdot (b - a)}{f(b) - f(a)}$$

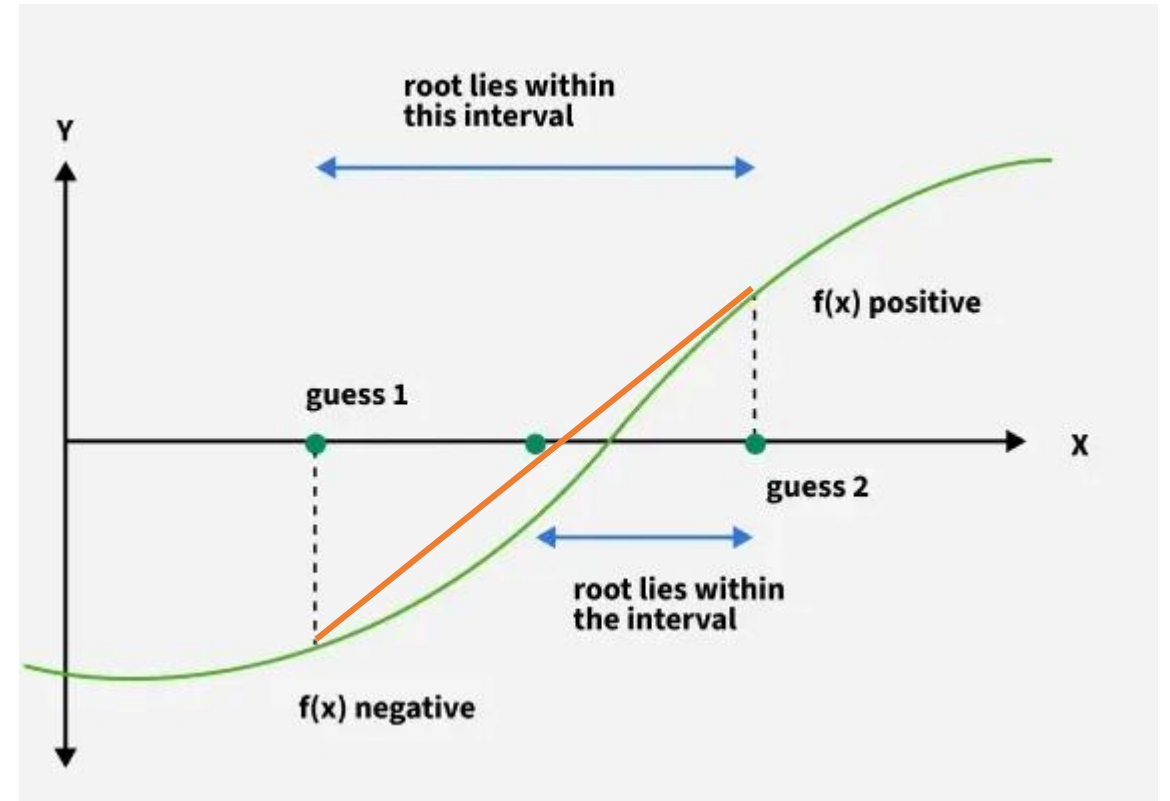
Iteration: The process is repeated by updating the interval based on the sign of $f(c)$:

$f(c) \cdot f(b) < 0$ Set $c = a$

Else if $f(c) \cdot f(a) < 0$ Set $c = b$

Pseudocode for Regula Falsi

1. Define the function $f(x)$
2. Choose initial guesses a and b such that $f(a) * f(b) < 0$
3. Set the tolerance (allowed error), ϵ
4. Repeat until $|f(c)| < \epsilon$:
 - (i) Compute $c = a - [f(a) * (b - a)] / [f(b) - f(a)]$
 - (ii). If $f(c) * f(a) < 0$: $b = c$ Else: $a = c$
7. Output c as the root



False Position

$$\frac{f(a)}{x_c - a} = \frac{f(b)}{x_c - b}$$

Solving:

$$x_c = b - \frac{f(b)(a - b)}{f(a) - f(b)}$$

You need to choose whether the new brackets are (a, x_c) or (x_c, b)

Advantage and Limitations

Ease of Use	Simple	Slower convergence compared to some methods
Robustness	Guarantees convergence if the function is continuous	May not find multiple roots effectively
No Derivatives	Does not require derivative calculations	Can be less efficient for certain functions
Convergence	?	

Regula Falsi Structural Flaws

⚠️ Asymmetric Bracket Updates

Unlike Bisection which halves the bracket length every step, Regula Falsi often updates only **one endpoint** continuously.

The bracket interval length does *not* tend to zero, rendering interval width useless as a stopping criterion!

🚦 Loss of Superlinear Speed

While theoretical secant methods achieve superlinear convergence, standard Regula Falsi often degenerates to **linear convergence**.

For steep or highly curved functions, Regula Falsi can perform even *slower* than simple Bisection!

End-Point Stagnation

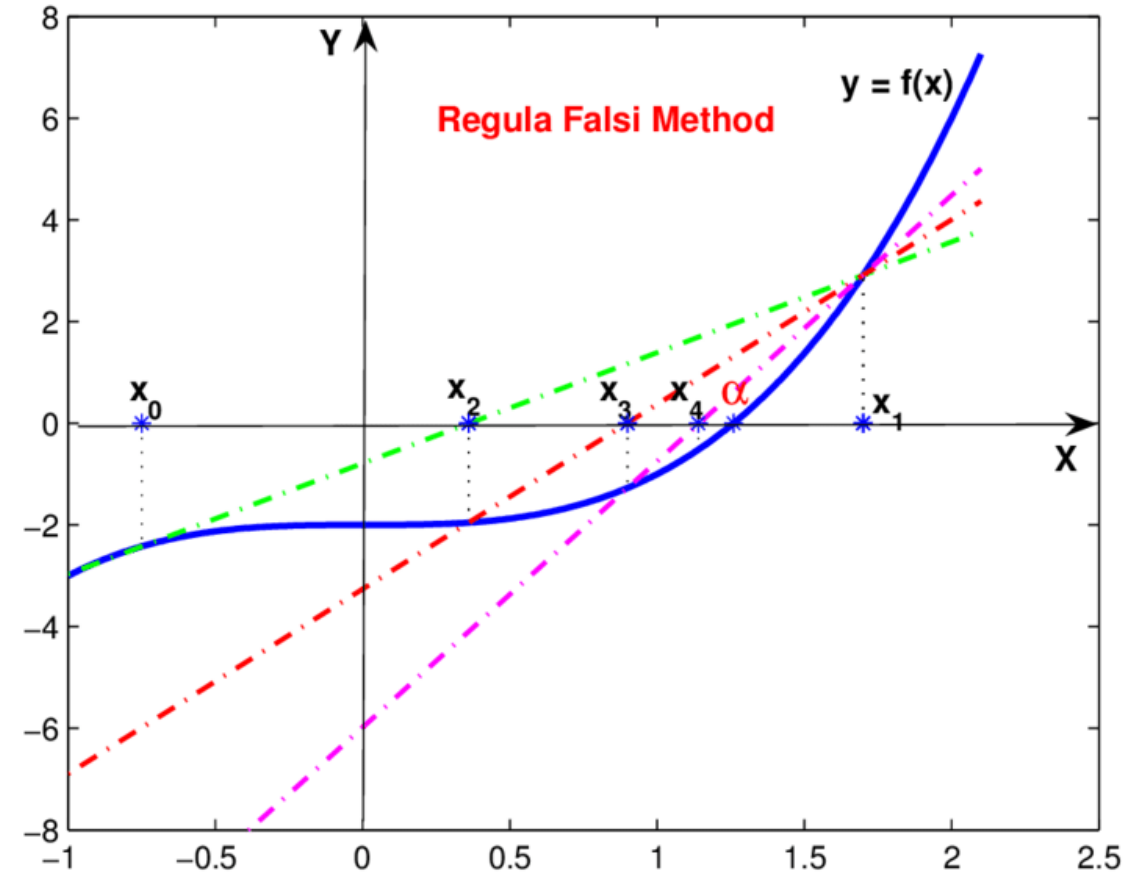
The Mechanics of Endpoint Freezing

When a function $f(x)$ is strictly convex ($f''(x) > 0$) or concave in the initial bracket $[a, b]$, the secant chord lies entirely above or below the curve.

As a result, the calculated x-intercept always falls on the **same side** of the true root. One endpoint (e.g. point a) remains permanently fixed ("stagnates"), while the other endpoint, x_c moves extremely slowly toward the root.

Example Function Slower than Bisection:

$$f(x) = x^{10} - 1 \quad \text{on} \quad [0, 1.3]$$



Defining Order of Convergence (r)

Mathematical Definition $\epsilon = x_n - a$

The order of convergence r measures how rapidly successive error estimates decrease toward zero:

$$\lim_{n \rightarrow \infty} \frac{|\epsilon_{n+1}|}{|\epsilon_n|^r} = C$$

Where $C > 0$ is the asymptotic error constant.

Values for Both Methods

Bisection Method ($r = 1$): Strictly linear with $C = 0.5$. The error strictly halves: $|\epsilon_{n+1}| = 0.5|\epsilon_n|$.

Ideal Regula Falsi ($r = 1.68$): Superlinear convergence (golden ratio rate) when both endpoints update continuously.

Stagnated Regula Falsi ($r = 1$): Degenerates to linear convergence with $C \rightarrow 1$, making error reduction extremely slow

Fixing Stagnation: Illinois Method

Detect Stagnation

If the same endpoint is selected twice in consecutive iterations, the algorithm flags the side as stagnant.

Half the Weight

The function value at the stagnant endpoint is artificially halved ($f(a) \leftarrow 0.5 \cdot f(a)$).

Restore Speed

This forces the next secant line to cross on the opposite side, breaking the freeze and achieving superlinear speed ($r \simeq 1.44$).

Method 3. Fixed Position

- Re-arrangement: Rewrite the equation $f(x) = 0$ as $x = g(x)$
- Start with an initial guess x_0
- Generate a sequence $x_{n+1} = g(x_n)$
- Keep checking for convergence- i.e if $\frac{|x_{n+1}-x_n|}{|x_n|} < \epsilon$, which means the value of x is not changing
- **Convergence Condition:** For the method to work, the function $g(x)$ should satisfy $|g'(x)| < 1$ near the solution, which helps ensure that the sequence converges.

Fixed Position- Algorithm

1. Define the function $g(x)$ such that $x = g(x)$
2. Choose an initial guess x_0
3. Set tolerance (epsilon) and maximum number of iterations $N_{\max}$
4. For n from 1 to $N_{\max}$:
 - (i). $x_{\text{next}} = g(x_{\text{current}})$
 - (ii). If $|x_{\text{next}} - x_{\text{current}}| < \text{epsilon}$:
return x_{next} as the root
 - (iii). $x_{\text{current}} = x_{\text{next}}$
5. If the root was not found after $N_{\max}$ iterations, report failure

ALGORITHM FixedPointIteration(g , x_0 , tol , max_iter):

INPUT: Iteration function g , initial guess x_0 , tolerance tol , max iterations max_iter

OUTPUT: Approximate root x , iteration history

$x = x_0$

history = [x_0]

FOR $k = 1$ TO max_iter DO

$x_{next} = g(x)$ # You need a function for $g(x)$

 history.append(x_{next})

IF $abs(x_{next} - x) < tol$ THEN

 RETURN x_{next} , history

Advantages and Limitations

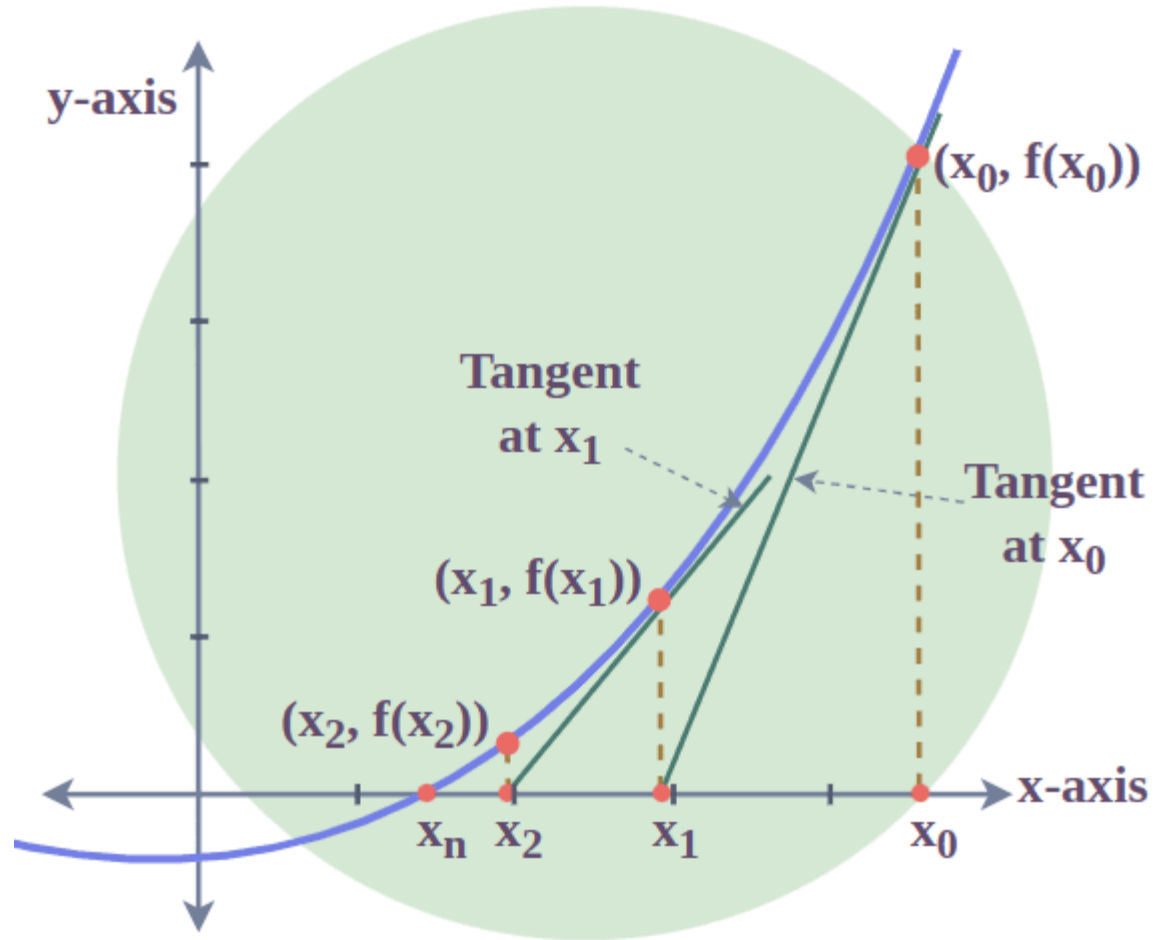
Advantages	Disadvantages
Simple	Slow convergence compared to Newton Raphson, Secant
No derivative needed	Conditional Convergence
Flexibility in function choice- you can write it using different functions $g(x)$	If you cannot write it using a good function $g(x)$ it may not converge- So check $ g'(x) $. Choice of x also matters
Low computational cost	Sensitive to initial conditions and function $g(x)$
	May fail silently

Method 4. Newton Raphson Method

1. Define the function $f(x)$ and its derivative $f'(x)$
2. Choose an initial guess x_0
3. Set tolerance epsilon for stopping criteria and maximum iterations $N_{\max}$
4. For n from 1 to $N_{\max}$:
 - a. Compute $f_{\text{val}} = f(x_n)$
 - b. Compute derivative $f'(x_n)$
 - c. If $f'(x_n) == 0$, stop (derivative zero, method fails)
 - d. Compute next approximation: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$
 - e. If $|x_{n+1} - x_n| < \text{epsilon}$ (or $|f(x_{n+1})| < \text{epsilon}$), return x_{n+1} as root
 - f. Else, set $x_n = x_{n+1}$ and continue
5. If maximum iterations reached without convergence, report failure or return latest approximation

Newton Raphson

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



Advantages

- Fast convergence- quadratic
- Single Initial Guess- no bracketing
- Efficient Iterative Process
- Applicability to Multidimensional Problems
- Can Refine Roots from Other Methods:

Limitations of the Newton-Raphson Method

- **Requires Derivative Calculation:** The method needs the function's derivative to be known and computable at each step, which may be difficult or expensive for some functions.
- **Dependency on Initial Guess:** Poor guesses can lead to failure to converge or divergence.
- **Fails at Zero Derivative Points:** If the derivative at the current guess is zero or near zero, the method either breaks down (division by zero) or leads to large jumps away from the root.
- **Poor Global Convergence:** Unlike bracketing methods, convergence is not guaranteed from an arbitrary initial point, and the method may converge to undesired roots or cycle indefinitely.
- **Trouble Near Local Maxima, Minima, or Inflection Points:** The tangent lines may poorly approximate the root direction, causing divergence or oscillation.
- **Slow or No Convergence for Multiple Roots:** The method converges slowly for roots with multiplicity greater than one.

Single vs. Multi-Variable Root Finding

Single Variable Scalar Form

Solves a single non-linear equation $f(x) = 0$ for a scalar unknown x using the first derivative.

$$x^{n+1} = x^n - \frac{f(x^n)}{f'(x^n)}$$

Uses tangent line slope to project step direction.

Multi-Variable Vector Form

Solves a system $F(x) = 0$ of m equations with m variables using the Jacobian matrix. At n th iteration :

$$x^{n+1} = x^n - \frac{F(x^n)}{J(x^n)}$$

Solves linear system $J(x) \Delta x = -F(x)$ at each iteration.

The Jacobian Matrix Structure

The **Jacobian Matrix** acts as the multidimensional linear derivative, mapping local displacement to function changes:

$$J(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Matrix must be non-singular ($\det(J) \neq 0$) to guarantee a unique step solution.

Execution Sequence Timeline

Step 2: Residual

Compute $F(x(k))$.
Check if residual
norm $\|F\| < \epsilon$ for
exit.

Step 4: Update

Update estimate
 $x(k+1) = x(k) + \Delta x$
and loop to next
step.

Step 1: Init

Set initial vector
 $x(0)$, tolerance ϵ ,
and max iterations
 $K_{\max}$.

Step 3: Solve

Compute $J(x(k))$ and
solve linear system $J \Delta x$
 $= -F$ via LU
decomposition.

Pseudocode & Algorithm Structure

// Multivariable Newton Raphson Pseudocode

```
function
MultivariableNewtonRaphson(F, J, x0,
tol, maxIter):
    x = x0 // Initialize state vector
    for k = 1 to maxIter:
        R = F(x) // Compute residual vector
        if norm(R) < tol:
            return x, "Converged (residual)"
        Jac = J(x) // Evaluate n x n Jacobian
        Δx = SolveLinearSystem(Jac, -R)
        x = x + Δx // State update
        if norm(Δx) < tol:
            return x, "Converged (step size)"
    return x, "Diverged / Max iterations"
```

Key Pseudocode Features

Vector Inputs: Operates on n -dimensional initial guess x_0 and vector function F .

Matrix Linear Solver: Avoids explicit matrix inversion by using LU or QR factorization for $Jac \cdot \Delta x = -R$.

Termination Controls: Guards against infinite loops via iteration threshold **maxIter**.

Dual Metric Check: Evaluates both residual vector norm $\|R\|$ and increment norm $\|\Delta x\|$.

Secant Method

1. Define the function $f(x)$
2. Choose two initial guesses x_0 and x_1 (close to the root)
3. Set tolerance epsilon for the stopping criterion and maximum iteration $N_{\max}$
4. For $i = 1$ to $N_{\max}$:
 - (i) Compute $f(x_0)$ and $f(x_1)$
 - (ii) If $f(x_1) == f(x_0)$, stop (division by zero)
 - (iii) Compute next approximation: $x_2 = x_1 - f(x_1) * (x_1 - x_0) / (f(x_1) - f(x_0))$
 - (iv) If $|x_2 - x_1| < \text{epsilon}$ (or $|f(x_2)| < \text{epsilon}$), return x_2 as the root
 - (v) Update $x_0 = x_1$ and $x_1 = x_2$ to proceed to next iteration
5. If maximum iterations reached without convergence, report failure or return last approximation

Comparison of Newton Raphson vs Secant

Aspect	Newton-Raphson Method	Secant Method
Algorithm Type	Uses derivative information explicitly	Uses two points to approximate derivative (no analytic derivative needed)
Initial Guesses	Requires a single initial guess	Requires two initial guesses
Derivative	Needs computation of the exact derivative $f'(x)$	Does not require derivative; approximates derivative from two points
Convergence Rate	Quadratic convergence (very fast if close to root)	Superlinear convergence (~ 1.618 , slower than Newton-Raphson)
Per Iteration Cost	Requires function and derivative evaluation	Requires two function evaluations per iteration but no derivative